GridSync MRP

Production build plan for **mrp_grid_scheduler** on Odoo 19 | 3-day solo build | Odoo UAE hackathon

One-line product. A tariff- and carbon-aware production scheduler for Odoo Manufacturing: it ingests real electricity price and carbon signals (live spot data where a market exists, contractual time-of-use tariffs where it does not), moves movable work orders to cheaper or cleaner windows without missing deadlines, and records what each work order actually cost and emitted.

Ground rule for the whole build. Every number on screen comes from a real source or from a user-entered input that carries its source reference. Nothing is simulated, hard-coded or presented as more certain than it is. Estimates are labelled as estimates.

1. What changed after the research (read this first)

Odoo UAE is a UAE audience, and the research showed the UAE has no day-ahead electricity market. That changes the product framing, not the architecture. The plan below is built around it.

Topic	What the sources say	Consequence for the build
UAE market	No liberalised wholesale market; large consumers buy under contractual tariffs, not day-ahead prices. ADDC lists a fixed tariff for transmission-connected customers from Jan 2023 to Dec 2027 and says it is intended to move to time-of-use later. ADDC also states that industrial customers above 1 MW connected load pay a higher peak tariff when summer daytime demand is highest.	A **time-of-use tariff provider is P0** and is the UAE use case. The European spot feed becomes the live-data proof and the export market. Peak in the UAE is summer daytime, the opposite of the European solar trough, which is a good demo contrast.
Energy-Charts price API	Keyless. Price endpoint limited to 2 requests/minute. Timestamps are ISO with offset and mark the start of each period (15-minute resolution). DE-LU prices are CC BY 4.0 (attribution required); other zones are restricted to private/internal use.	Fetch on a cron, cache in the database, never call per click. Show an attribution line. Use DE-LU only.
/co2eq endpoint	The deep-research report cites it, but it is not in the published endpoint catalog (price, public_power, ren_share, signal, ...).	Derive carbon intensity as an **estimate** from public_power or ren_share_forecast with editable factors. Label it. Test /co2eq once, but do not depend on it.
UK Carbon Intensity API	Half-hour national and regional forecasts (fw48h, postcode variants); 'actual' exists only for past periods. No price data.	P1 provider for carbon-only mode (GB). Do not mix UK carbon with German prices in one run.
Odoo ESG app	Enterprise-only in Odoo 19.	ESG bridge is an optional extra module (P2). Core module must not depend on it.
Existing Odoo apps	A third-party Odoo 19 module already links manufacturing orders and machines to electricity emission factors.	Do not pitch 'carbon in MRP' as new. Pitch 'schedule moves driven by live tariff and carbon signals'. Re-search the store before the pitch; absence is not proven.
Odoo 19 API	models.Constraint replaces _sql_constraints; groups use privilege_id and users use group_ids; record._cr/_context/_uid are deprecated; odoo.fields.Domain replaces odoo.osv.	Write 19-native code from the start (section 8).
Savings numbers	The report's 82% is one hand-picked job moved from peak to trough; 12-18% is vendor marketing.	Never quote either. Show savings computed from your own runs, next to a lower bound (section 5).

2. What 'production-level' means for a 3-day build

Three days for one person is tight for the full vision, so 'production-level' here means **a smaller surface built to a real standard**, not a large surface built loosely. The priority tiers in section 6 are designed so that P0 alone is a complete, shippable product.

- **Correct:** the optimizer is pure Python with unit tests covering DST changes, negative prices, infeasible deadlines, precedence and unit conversions.
- **Safe:** nothing changes a work order without explicit confirmation, and every apply stores a snapshot so it can be reverted.
- **Idempotent:** re-running a sync or an optimization never duplicates or corrupts data (unique constraints, upserts, immutable run snapshots).

- **Honest:** stale, forecast and estimated data are flagged in the UI. Nothing is labelled 'audit-ready' or 'certified Scope 2'. It is an hourly estimate for planning.
- **Secure and multi-company:** access groups, company_id on every model, HTTPS to allow-listed hosts only, timeouts on every request, no secrets in code.
- **Observable:** a sync log and run log show failures and data age to the planner.
- **Installable and upgrade-safe:** installs on a fresh Odoo 19 database, no monkey-patching of core, tests pass in CI.

3. Architecture and scalability

Five layers, each replaceable on its own: **Providers** (fetch and normalise) > **Slot store** (one table of price and carbon per interval) > **Optimizer engine** (pure Python, no ORM, deterministic) > **Apply layer** (writes to work orders with snapshot and revert) > **Actuals ledger** (what really ran, P1). The optimizer never touches the database, which makes it testable, fast and reusable.

Concern	Design decision
More data sources later	Provider registry: each provider type implements one fetch method returning normalised rows (start_utc, minutes, price, carbon, flags). Adding a market means adding one class, not touching the optimizer.
Many work orders	Optimizer cost is roughly orders x slots using prefix sums. Run synchronously up to a configurable threshold; above it the run goes to state 'queued' and a cron processes it in batches with progress commits (Odoo's cron progress API; verify exact call in 19).
Large slot tables	Unique index on (provider, start_utc); bulk upsert per fetch; retention cron archives slots older than N days; the optimizer reads only its horizon window.
Mixed units and currencies	Price stored as per-MWh in the provider's own currency, carbon as gCO2e/kWh. A run uses one provider per workcenter group; never mix currencies inside one total. Conversion to company currency via Odoo rates is a P1 display feature.
Multiple sites	Provider is linked per workcenter, so plants in different zones or tariffs work. Runs are per company.
Failure of external APIs	Retry with backoff, store raw payload hash, keep serving cached slots, show a stale-data banner. The optimizer refuses to fill gaps: jobs that fall outside covered slots are marked infeasible with a reason.
Concurrent users	Runs are immutable snapshots; apply checks that the work order has not changed since the snapshot and aborts that line if it has.
Future solver upgrade	The engine has a small interface (inputs dataclasses in, placements out), so an exact MILP/CP solver can replace the heuristic later without changing views or models (P2).

4. Data model

Model	Purpose	Key fields and constraints
grid.provider	A source of tariff and carbon data	name, type (energy_charts / uk_carbon / manual_tou / csv), zone_code, currency_id, source_url, license_text, attribution, resolution_min, valid_from/valid_to, last_sync_at, state, company_id
grid.tariff.profile + rule	Manual time-of-use schedule (the UAE case)	weekday set, start/end time, season date range, price_per_kwh, optional carbon factor, source reference. A generator expands rules into slots so the optimizer only ever sees slots.
grid.tariff.slot	One row per provider per interval	provider_id, start_utc (indexed), minutes, price_per_mwh, co2_g_kwh, co2_source (published / estimated / manual), is_forecast, fetched_at. Unique (provider_id, start_utc).
grid.sync.log	Audit of every fetch	provider, started_at, duration_ms, http_status, rows_inserted, rows_updated, error, payload_hash
mrp.workcenter (ext.)	Machine energy profile	power_draw_kw, setup_power_kw, grid_provider_id, allow_grid_shift, shift_penalty_per_hour
grid.optimization.run	One optimize click	state (draft, queued, computing, done, failed, applied, reverted), horizon, carbon_price_per_t, provider, baseline and proposed totals (cost and kgCO2e), lower_bound_cost, data_stale flag, company_id, user_id
grid.optimization.line	Before/after per work order	workorder_id, baseline_start/end, proposed_start/end, kwh, baseline_cost, proposed_cost, baseline_kg, proposed_kg, status (shifted / unchanged / infeasible), reason
grid.energy.entry (P1)	Actuals per work order per slot	workorder_id, slot_start, kwh, price, carbon, cost, kg, analytic_line_id. Unique (workorder_id, slot_start) so re-posting is safe.

5. Optimizer specification

Objective: minimise energy cost plus a monetised carbon term, with a displacement penalty. The slider is an internal carbon price (currency per tonne CO2e), which is easier to defend than a unitless weight.

```
slot_cost[t] = kW * (g/60) * ( price[t]/1000 + carbon_price * CI[t]/1,000,000 )
  g = slot length in minutes, price in currency/MWh, CI in gCO2e/kWh, carbon_price in currency per tCO2e
job_cost(start) = sum of slot_cost over the job's slots + shift_penalty * hours_displaced
```

Inputs are read once into plain dataclasses (no ORM inside the solver): movable work orders (not started), duration (duration_expected, minutes), power, workcenter, earliest start, deadline (from the MO), predecessors, and fixed obstacles (in-progress or locked work orders).

Algorithm (P0)

- Resample slots to a common grid. Check coverage of the horizon; never extrapolate. Uncovered jobs become 'infeasible: insufficient data'.
- For each job compute the cost of every feasible start in one pass using prefix sums.
- Order jobs topologically within each MO (successor starts after predecessor finishes), then earliest deadline first across each workcenter.
- Place each job at the cheapest feasible start that does not overlap fixed or already-placed jobs on that workcenter. Ties go to the earliest start for stability.
- Run a bounded local-improvement pass (shift/swap adjacent jobs on a workcenter if cost drops and constraints hold).
- Compute a **lower bound**: each job placed independently at its cheapest window, ignoring overlaps. Report the gap between your result and the bound. This is the honest answer to 'is it optimal?'. It is a heuristic, and the UI says so.
- Deterministic: identical inputs always give identical placements.

Edge cases that need tests: DST transitions (23 and 25 hour days); negative prices (the optimizer should seek them); partial slots at job edges; forecast versus final prices; jobs longer than the horizon; zero-power workcenters; deadline earlier than any feasible finish; a work order started or edited after the run was created.

Carbon intensity for DE-LU is an estimate. Generation mix times per-technology emission factors, or the renewable-share forecast as a forward proxy. Factors are editable records with a source field. Start from IPCC lifecycle medians and verify the values before use. For the UAE no public hourly intensity feed was found; use a user-entered factor or profile with its source, flagged 'manual'.

6. Priorities: what to finish first, and what is extra

P0 - must finish (shippable on its own)		
#	**Deliverable**	**Done when**
1	Module skeleton, access groups (privilege_id), multi-company, manifest, CI test run	Installs on a fresh DB; tests run green
2	Provider framework + grid.tariff.slot + sync log	Unique constraint holds; re-sync is idempotent
3	Energy-Charts DE-LU price fetcher (v2 /price), ISO to UTC conversion, cron with retry and cache	Live slots stored; works with network off; DST test passes
4	Manual time-of-use profile + slot generator + CSV import, with real utility tariff values and source URL	Next 7 days of slots generated from the published tariff
5	Carbon: estimated DE-LU intensity + manual factor/profile, always labelled with its source type	co2_source visible on every slot
6	Workcenter power fields and provider link	Fields in form and list views
7	Optimizer engine (pure Python) + unit tests + lower bound	All edge-case tests pass
8	Run + line models, snapshot loader, wizard, apply and revert on work orders	Optimize, apply, revert works on a real MO in the UI
9	Run form with before/after table, KPIs, stale-data banner, pivot/graph	Planner can read the result without help

P1 - finish if P0 gates are green (Day 3 morning)		
#	**Deliverable**	**Done when**
10	Actuals ledger: on work-order completion compute energy per slot from time logs; unique per work order and slot	Re-finishing does not double-count
11	Analytic posting of electricity cost (verify analytic setup in Odoo 19 first)	Analytic line linked to the ledger entry
12	Constraints: workcenter calendar and shift-overtime penalty	Penalty changes proposals as expected in tests
13	UK Carbon Intensity provider (carbon-only mode)	GB half-hour slots stored with forecast flag
14	Currency display in company currency; README; install and upgrade test; 2,000-order load test (synthetic, never shown in the demo)	Run completes within a stated time budget

P2 - extras (only after the Day 3 feature freeze gate)		
#	**Deliverable**	**Done when**
15	Optional bridge module for Odoo ESG (Enterprise only), writing hourly estimates	Separate module; core unaffected
16	Odoo AI agent tool wrapper for the optimizer (verify the 19 AI API first; the report's agent claims were for Odoo 20)	Natural-language command triggers a real run
17	Exact MILP/CP-SAT solver behind the same interface; gap-to-bound reporting	Solver installs cleanly; falls back to heuristic
18	Gantt view (Enterprise), rolling re-optimization cron, PPA/contract tariff records, on-site solar/battery inputs	Each is independent

Explicitly not building: audited Scope 2 or 24/7 CFE claims, fabricated historical data, a custom forecasting model, anything that needs credentials you do not have. If a feature cannot be backed by a real source in 3 days, it stays out of the demo.

7. Three-day schedule

Day	Block (approx. hours)	Work	Exit gate
1	0-1	Environment (confirm Community vs Enterprise on the event instance), repo, manifest, groups	Module installs
1	1-4	Provider, slot, sync-log models; constraints; access rules	Models and tests green

Day	Block (approx. hours)	Work	Exit gate
1	4-7	Energy-Charts fetcher, parser, fixtures saved from real responses, cron + retry + stale flag	Real DE-LU slots in DB; offline run works
1	7-10	Manual TOU profile and slot generator with real utility tariff; CSV import; estimated carbon	7 days of TOU slots; all slots labelled by source
2	0-1	Workcenter fields and views	Fields visible
2	1-5	Optimizer engine and unit tests	Tests incl. DST, negative price, infeasible pass
2	5-8	Run/line models, snapshot loader, wizard, apply/revert; verify workorder date write behaviour in 19	End-to-end optimize/apply/revert on a real MO
2	8-11	Run form, KPIs, graph/pivot, chatter message; real demo MOs entered as business data	Planner-readable result screen
3	0-5	P1: actuals ledger, analytic posting, penalty and calendar, clean-DB install test, README, load test	Feature freeze at hour 5-6
3	5-8	Hardening: security pass, error messages, stale-data paths, bug fixes. P2 only if every gate above is green	Release candidate
3	8-10	Demo rehearsal x2, backup plan, final install test	Demo runs start to finish

8. Odoo 19 implementation notes

- Constraints as model attributes: **models.Constraint**, not _sql_constraints. Groups via **privilege_id**; users via **group_ids**.
- Use **self.env.cr / self.env.context / self.env.uid**; the underscore versions are deprecated. Use **odoo.fields.Domain**, not odoo.osv.
- Views: **<list>** not <tree>; conditions as inline expressions (invisible="state != 'done'"), no attrs. Translations via self.env._().
- Work-order dates: in the 18.0 source, date_start and date_finished are computed from the linked calendar leave and have an inverse. Write the dates through the fields, then confirm in the 19 source and with a test that the leave and the MO planning dates update. Do not write leave records directly.
- Gantt is an Enterprise view. Core deliverable uses list, pivot, graph and the run form so it works on Community as well.
- All timestamps: parse ISO with offset, convert to naive UTC for storage, display in the user's timezone. Never store local times.
- HTTP: requests with a timeout on every call, HTTPS only, host allow-list, a descriptive User-Agent, honour Retry-After on 429.
- Licence header and manifest: LGPL-3 for an Odoo Store app, correct depends (mrp only for core), version 19.0.x.y.z.

9. Quality gates

Gate	Check
Parser tests	Recorded real Energy-Charts responses are stored as test fixtures (fixtures are not fake data; the live demo still hits the live API). Cover DST days and null values.
Optimizer tests	Known small cases with hand-computed answers; property test: result never violates deadline, overlap or precedence; lower bound is never above the result.
Idempotency	Run sync twice, optimize twice, apply twice: no duplicates, no double counting.
Revert	Apply then revert restores the original dates exactly.
Security	A user without the planner group cannot apply or change providers; cross-company access blocked; no sudo without a comment explaining why.
Clean install	Fresh database install and uninstall without errors; no demo data needed for the module to load.
Failure paths	Network off, 429 response, empty payload, partial payload: each shows a clear message and keeps the last good data.

10. Demo plan (2 to 3 minutes, all real)

- **Track A, UAE:** a heat-treatment furnace workcenter on the real utility time-of-use tariff (values entered with the tariff document URL). Optimize; the job moves out of the peak window; show before/after cost and the lower bound.
- **Track B, live market:** switch the workcenter to the DE-LU provider. Show slots fetched from the live API, with attribution and age. Move the carbon-price slider and show the placement change.
- **Resilience:** cut the network, re-run from cached slots, show the stale-data banner.
- **Numbers rule:** quote only what the screen shows from that run. Say 'heuristic, with gap to lower bound X%' when asked about optimality.
- **Likely questions:** 'Does it work in the UAE?' (yes, via contractual time-of-use tariffs; ADDC signals a move to time-of-use for transmission customers). 'How accurate is the carbon figure?' (estimate, method and factors visible and editable). 'Who pays?' (plants with time-varying tariffs; validate before claiming a market size).

11. Risks and things to verify

Risk or unknown	Mitigation
Scope versus 3 days	Tier discipline. Feature freeze at Day 3 hour 5-6. P0 is complete on its own.
UAE tariff values	I could not confirm a published industrial time-of-use schedule in the searches. Read the utility's current tariff document, enter the real values, store the URL and validity dates. If only a flat tariff applies to your demo customer, say so and demo Track B as the main proof.
Carbon credibility	Label as estimate, show method, keep factors editable; offer carbon-only comparison rather than claiming audited figures.
Energy-Charts availability and limits	Cache first; fixtures for tests; stale banner; fetch once per cron cycle.
Work-order date writes in 19	Verify on Day 2 morning with a test before building the apply layer on top of it.
Event instance may be Community or Enterprise	Core avoids Enterprise-only dependencies; ESG and Gantt are optional.
Submission format unknown	Ask the organisers whether they want a repo, a hosted instance or both, and whether outbound internet is allowed on the venue network.

Sources

- Energy-Charts API endpoint catalog, limits and licence: https://api.energy-charts.info/llms.txt
- ADDC 2025 tariff (transmission-connected fixed tariff, intended move to time-of-use): https://www.addc.ae/en-US/business/Documents/2025%20Tariff%20(English).pdf
- ADDC business rates and tariffs 2025 (industrial above 1 MW, peak tariff): https://www.addc.ae/en-US/business/Pages/RatesAndTariffs2025.aspx
- UAE power sector overview 2026, no liberalised wholesale market: https://gpg-pdf.chambers.com/power-generation-transmission-and-distribution-2026/352/
- UK Carbon Intensity API endpoints (client library reference): https://beta.pkg.go.dev/github.com/evcc-io/evcc/tariff/ngeso
- Odoo ORM changelog: https://www.odoo.com/documentation/saas-19.1/developer/reference/backend/orm/changelog.html
- Odoo 19 migration notes (models.Constraint, privilege_id, group_ids): https://github.com/ledoent/dms/pull/1
- Odoo 18.0 mrp_workorder source (date fields and inverse): https://sources.debian.org/src/odoo/18.0.0%2Bdfsg-2/addons/mrp/models/mrp_workorder.py
- Odoo 19 ESG is Enterprise-only: https://website.technaureus.com/blog-detail/esg-module-in-odoo-19
- Existing third-party manufacturing carbon module: https://apps.odoo.com/apps/modules/19.0/carbon_emission_management

Items marked 'verify' were not confirmed against Odoo 19 source or official documentation during research and must be checked before the build relies on them.